

边缘 AI Agent 网关—快速上手文档

目标：两天内能讲清楚这个项目的每个细节，面试不虚。

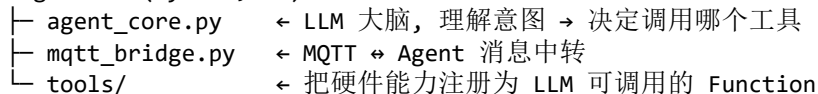
一、先记住这 3 张图

1.1 系统架构（4 层）

用户（自然语言 / Web Chat）



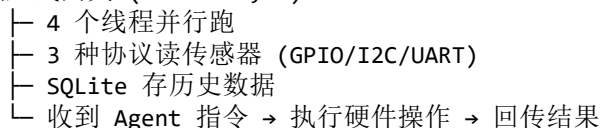
AI Agent 层 (Python, PC)



MQTT (mosquitto broker)



嵌入式网关 (i.MX6ULL, C)



硬件层 (DHT11 / BH1750 / PMS5003 / LED / 继电器 / 红外)

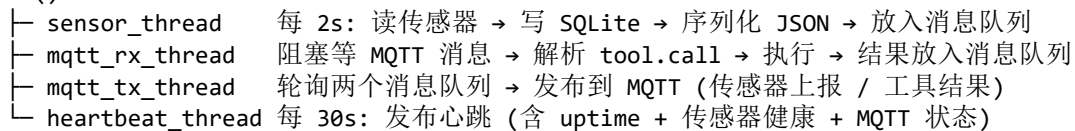
1.2 数据流（一条指令的完整路径）

用户说 "打开客厅灯"

- Agent 调用 LLM, LLM 返回 function_call: control_led
- Agent 通过 MQTT 发布到 edge/{device_id}/tool/call
- 嵌入式网关 mqtt_rx_thread 收到消息
- tool_dispatcher 解析 JSON → 匹配 "control_led" → 调 device_led_set()
- gpio_write_val(GPIO2, 1) → LED 亮
- 结果序列化为 JSON → MQTT 回传 edge/{device_id}/tool/result
- Agent 收到 → LLM 生成回复 "客厅灯已打开"
- 用户看到回复

1.3 4 线程架构 (main.c)

main()



线程间通信: POSIX 消息队列 (mq_open/mq_send/mq_receive)

为什么不用共享内存? → 消息队列自带同步, 不需要额外加锁

为什么不用管道? → 消息队列有边界, 支持优先级, 更适合离散消息

二、文件地图（按面试重要性排序）

必看文件

文件	行数	核心内容	面试能聊什么
main.c	305	4 线程架构、消息队列、初始化流程	多线程设计、线程间通信选型
sensor_manager.c	215	统一管理传感器，读写缓存，JSON 上报	传感器融合、容错设计
mqtt_client.c	152	libmosquitto 封装，工具注册表，retained 消息	MQTT 原理、QoS、retained
tool_dispatcher.c	147	JSON 解析，查表分发，工具注册表	Agent Function Calling 原理

重点文件

文件	行数	核心内容	面试能聊什么
pms5003.c	135	UART + epoll 读 PM2.5 传感器帧	epoll vs select/poll、帧解析、校验和
bh1750.c	100	I2C ioctl 读光照传感器	I2C 协议、内核调用链
w25q32.c	116	SPI 全双工传输，JEDEC ID 读取	SPI 4 种模式、全双工原理
sqlite_storage.c	259	WAL 模式，参数化查询，数据清理	SQLite 选型理由、WAL vs DELETE
ir_control.c	101	NEC 红外协议，38kHz 载波 bit-banging	红外编码格式、载波调制

浏览即可

文件	内容
config.h	所有宏定义（GPIO 引脚/I2C 地址/MQTT 主题/采样周期）
protocol.h	JSON-RPC 协议结构体定义
dht11.c	DHT11 GPIO bit-banging（简化实现）
led_control.c	GPIO sysfs 控制 LED
relay_control.c	GPIO sysfs 控制继电器
gpio_util.c	GPIO export/write 基础操作
mqtt_ringbuf.c	环形缓冲区（生产者-消费者）
tool_handlers.c	6 个工具函数实现
status_led.c	状态灯（占位实现）
logging.c	自研日志系统（5 级/线程安全/颜色）

三、核心数据结构（看完这些就懂了协议层）

```
// Agent → 设备：工具调用
typedef struct {
    char id[32];           // "req_001" 请求-响应匹配用
    char tool[32];         // "control_led"
    char args[256];        // {"device":"living_room_light","action":"on"}
} tool_call_t;
```

```
// 设备 → Agent: 执行结果
typedef struct {
    char id[32];           // 对应请求 ID
    int success;           // 0= 失败 1= 成功
    char data[512];        // JSON 结果
} tool_result_t;

// 传感器上报
typedef struct {
    char name[32];         // "temperature"
    double value;          // 26.5
    char unit[16];         // "celsius"
} sensor_value_t;

typedef struct {
    char id[32];
    char timestamp[32];    // "2026-07-01T12:30:00"
    int count;
    sensor_value_t sensors[8]; // 最多 8 种传感器
} sensor_report_t;
```

四、MQTT Topic 速查

Topic	方向	干什么
edge/{id}/register	设备 → Agent	设备启动时发布工具注册表 (retained)
edge/{id}/sensor/report	设备 → Agent	每 5s 上传传感器数据
edge/{id}/tool/call	Agent → 设备	Agent 下发指令
edge/{id}/tool/result	设备 → Agent	设备回传执行结果
edge/{id}/heartbeat	设备 → Agent	每 30s 心跳

五、6 个硬件驱动速查

DHT11 温湿度 (GPIO)

协议: 单总线, 主机拉低 18ms → DHT11 回 40bit (湿度8+湿度8+温度8+温度8+校验8)

代码: dht11.c — 简化版, 实际 bit-banging 部分是伪代码

面试: "为什么不用内核驱动?" → 教学目的展示协议原理, 生产用内核驱动或 hrtimer

BH1750 光照 (I2C)

协议: I2C, 地址 0x23

流程: write(0x10 测量指令) → usleep(180ms) → read(2 字节) → lux = raw / 1.2

代码: bh1750.c 的 i2c_open() 用 ioctl(I2C_SLAVE, addr) 设从机地址

面试: open→i2cdev_open→ioctl→i2c_master_send, read 时内核自动加 R/W 位

PMS5003 PM2.5 (UART + epoll)

协议: UART 9600bps 8N1, 主动上报 32 字节帧

帧格式: 0x42 0x4D + 帧长 + PM1.0 + PM2.5 + PM10 + ... + 校验和

代码: 用 epoll 等待帧头 0x42 0x4D, 前 30 字节累加验证校验和

面试: 为什么 epoll 不用 select? O(1) vs O(n), 无 fd 数限制

W25Q32 SPI Flash (SPI)

协议: SPI Mode 0, 10MHz

JEDEC ID 读取: 发 [0x9F, 0xFF, 0xFF, 0xFF] → 收 [垃圾, 制造商, 类型, 容量]

代码: w25q32.c 的 spi_transfer() 用 SPI_IOC_MESSAGE ioctl

面试: 为什么发 4 字节只收到 3 字节有效? SPI 全双工, 发命令那字节收的是垃圾

LED / 继电器 (GPIO sysfs)

原理: 写 /sys/class/gpio/gpioN/value → 1/0, 内核 gpio 子系统处理

面试: sysfs 适合低速 GPIO, 高速用 /dev/mem + mmap

红外发射 (GPIO bit-banging, NEC 协议)

载波: 38kHz (26us 周期, 1/3 占空比)

引导码: 9ms 载波 + 4.5ms 空闲

逻辑 0: 560us 载波 + 560us 空闲

逻辑 1: 560us 载波 + 1690us 空闲

代码: ir_carrier_us() 循环翻转 GPIO

面试: usleep 精度不够, 生产用硬件 PWM 或内核 hrtimer

六、面试 20 题速答

Q1: 这个项目做什么的?

把 AI Agent 的能力延伸到嵌入式设备上。用户说”开灯” → LLM 理解意图 → 调 control_led 工具 → MQTT 下发指令 → i.MX6ULL 执行 GPIO 操作 → 灯亮。从”被动执行指令”升级到”主动理解意图”。

Q2: 为什么用 MQTT 而不是 HTTP?

1) 发布/订阅解耦 (设备不关心谁在消费数据); 2) MQTT 协议头最小 2 字节, HTTP 几百字节, 嵌入式带宽宝贵; 3) QoS 三级保证; 4) 遗嘱机制 (设备掉线自动通知)

Q3: QoS 1 和 retained 消息什么意思?

QoS 1: 至少送达一次, broker 会重发直到收到 PUBACK。retained: broker 保存最后一条消息, 新订阅者立即收到, 用于设备注册表 (Agent 重连后自动获取设备能力)。

Q4: 多线程怎么设计的?

4 线程: sensor(采集 + 存 SQLite)、mqtt_rx(收指令 + 分发)、mqtt_tx(消费消息队列 → 发布)、heartbeat(心跳)。线程间用 POSIX 消息队列通信, 用完即丢, 天然同步。

Q5: 为什么用消息队列不用共享内存?

消息队列有边界和优先级, 自带同步不需额外锁。共享内存需要信号量配合, 复杂且易出错。传感器数据 2s 一次, 频率低, 消息队列性能完全够。

Q6: epoll 比 select/poll 好在哪里?

1) O(1) 事件通知, select 要 O(n) 遍历所有 fd; 2) 无 fd 数量限制 (select 默认 1024); 3) 事件驱动 (回调), 不是轮询; 4) 用红黑树 + 就绪链表实现

Q7: I2C 通信流程讲一下?

open(“/dev/i2c-1”) → ioctl(I2C_SLAVE, 0x23) → 内核保存从机地址 → write() 发测量指令 → 内核自动加 R/W 位 → SCL/SDA 硬件发 → read() 内核再发读命令 → 收到 2 字节 → 计算 lux

Q8: SPI 全双工是什么意思?

主机发时钟 → 发 1 bit 数据 → 同时收 1 bit 数据。所以读 N 字节必须同时发 N 字节 (dummy)。
JEDEC ID: 发 [cmd, 0xFF, 0xFF, 0xFF] 4 字节, 收 [垃圾, ID1, ID2, ID3] 4 字节。

Q9: SPI 有哪 4 种模式?

CPOL(时钟空闲电平) + CPHA(采样边沿): Mode0(CPOL=0, CPHA=0) 空闲低, 上升沿采样。W25Q32 用 Mode0。

Q10: DHT11 的校验怎么做的?

40bit 数据: 湿度整数 + 小数 + 温度整数 + 小数 = 校验和 (8bit)。if(data[0]+data[1]+data[2]+data[3] != data[4]) 丢弃。

Q11: DHT11 bit-banging 为什么说实际不能用?

usleep 精度约 10ms 级别, DHT11 位时序要求 us 级别。内核用 udelay(忙等) 或 hrtimer(纳秒精度) 才能准确。教学展示协议原理用。

Q12: Agent Function Calling 怎么映射到硬件?

Agent 定义 6 个工具 (control_led/control_relay/send_ir/read_temperature/read_humidity/get_status)。
LLM 决定调哪个 → Agent 发 MQTT → 嵌入式 tool_dispatcher 做 strcmp 查表 → 调对应 C 函数 → 结果回传。

Q13: 为什么用 SQLite 不用纯文件?

文件只能顺序读写, SQLite 支持 SQL 查询 (WHERE/ORDER BY/AVG)。零配置, 单文件, 不需要独立进程。WAL 模式读写不互斥, 采集写时 Agent 也能读。

Q14: WAL 模式是什么?

Write-Ahead Logging: 写操作先写 WAL 日志文件 (append-only), 不阻塞读。读操作直接读主数据库。定期 checkpoint 把 WAL 合并到主库。对比 DELETE 模式 (写阻塞读)。

Q15: 为什么用参数化查询 (? 占位符)?

防 SQL 注入 + 预编译提升性能 (sqlite3_prepare 一次, 多次 bind+step)。

Q16: 环形缓冲区满了怎么办?

当前: 丢弃新消息 (不阻塞 mosquitto 网络线程)。生产环境: 用 pthread_cond 让消费者等待, 或动态扩容链表。

Q17: 程序怎么安全退出的?

signal_handler 收到 SIGINT/SIGTERM → 设 g_running=0 → 4 个线程各自退出循环 → main 里 pthread_join 等它们结束 → 依次清理 MQTT/GPIO/SQLite → 删消息队列

Q18: 日志系统怎么设计的?

5 级 (ERROR/WARN/INFO/DEBUG/TRACE), 编译期过滤 (低于 LOG_LEVEL 的宏展开为空, 零开销), 线程安全 (pthread_mutex), 毫秒时间戳 (gettimeofday), 彩色终端, 支持 HEXDUMP

Q19: 交叉编译怎么做的?

Makefile 设 CROSS_COMPILE=arm-linux-gnueabihf-, -mcpu=cortex-a7 -mfpu=neon。本机测试 make test-local 用 gcc。

Q20: 这个项目有什么不足/可以改进的?

- 1) DHT11 bit-banging 是伪代码, 实际该用内核驱动; 2) JSON 解析手写的 strstr, 该用 cJSON; 3) 环形缓冲区没加锁; 4) 部分工具函数返回硬编码值; 5) 没有单元测试; 6) 没有 OTA 升级机制。

七、两天学习路线

第 1 天上午: 走通数据流 (2h)

1. 打开 main.c, 看懂 4 个线程的启动和退出
2. 打开 protocol.h, 记住 tool_call_t / tool_result_t / sensor_report_t 三个结构体
3. 随便选一条 MQTT Topic, 在脑子里模拟 "Agent 下发 → 设备执行 → 结果回传" 的整条链路

第 1 天下午: 啃硬件驱动 (3h)

1. bh1750.c — 最短最完整, I2C 协议范例, 先看这个
2. pms5003.c — UART + epoll, 帧解析, 校验和验证
3. w25q32.c — SPI 全双工, 为什么发 4 收 3
4. dht11.c + ir_control.c — bit-banging, 知道原理和局限性就行

第 2 天上午: 消化软模块 (2h)

1. tool_dispatcher.c — JSON 解析 + 查表分发 + 工具注册表
2. sqlite_storage.c — WAL + 参数化查询 + 数据清理
3. mqtt_client.c — libmosquitto 回调 + retained + 离线遗嘱
4. mqtt_ringbuf.c — 环形缓冲区实现

第 2 天下午: 模拟面试 (2h)

1. 把上面 20 道题自问自答一遍, 要求每道题 1 分钟内讲清楚
2. 画架构图: 4 层 + 4 线程 + 5 个 MQTT Topic
3. 准备一个 STAR 故事: 遇到的困难 + 怎么解决的 + 学到了什么
(比如: bit-banging 精度不够 → 了解到用户态实时性的局限 → 学会区分教学代码和生产代码)

八、面试 STAR 故事模板

Situation: 我在做一个边缘 AI 网关, 需要把 DHT11/BH1750/PMS5003 三个传感器的数据汇总上报, 同时接收 Agent 的指令控制 LED 和继电器。

Task: 核心挑战是: 三种传感器用三种不同协议 (GPIO/I2C/UART), 不能让任何一个传感器的慢速读取堵住整个系统。

Action: 我设计了多线程架构: 传感器采集一个线程 (用 epoll 非阻塞读 PMS5003, 读失败用上次缓存值), MQTT 收发各一个线程, 心跳一个线程。线程间用 POSIX 消息队列通信, 不用共享内存 (避免加锁复杂度)。传感器数据先写 SQLite (WAL 模式, 读写不互斥), 再通过消息队列异步上报。

Result: 系统可以同时处理传感器采集和 Agent 指令, 采样周期稳定在 2s。SQLite 保留 7 天历史数据, 定时清理防止磁盘写满。

九、关键词清单 (简历/自我介绍用)

嵌入式 Linux (i.MX6ULL Cortex-A7) | POSIX 多线程 | 消息队列
I2C / UART / SPI 三种总线协议 | epoll I/O 多路复用
MQTT (libmosquitto) | QoS 1 | Retained 消息 | JSON-RPC 协议
SQLite3 + WAL 模式 | 参数化查询

GPIO sysfs | bit-banging (DHT11 / NEC 红外)
AI Agent Function Calling | LLM 工具调度
交叉编译 (arm-linux-gnueabihf) | Makefile / CMake

记住：面试官不关心你写了多少行代码，关心你能不能 1 分钟内讲清楚”这东西怎么工作的”和”为什么这样设计”。